ClaudeCode 你想知道的所有秘密，源码深度研究报告

Xiao Tan
@tvytlx · 16h ·

Follow

41 506 2.3K 426K

最近 Claude Code 泄露了 map 文件，导致泄露了源码文件，于是我下载了 npm 包里的 cli.js.map，从里面提取出 4756 个源码文件，沿着入口、提示词、工具、权限、Agent 调度、插件、Hook 一路拆下去，我发现其实 Claude Code 的 system prompt 只是冰山露出水面的一小块。

这篇文章会把冰山下面的部分摊开。你不需要是做 Agent 产品的工程师，也能从里面看到一套通用的系统设计思路。看完之后，你再去看市面上任何一个 AI Agent 产品，会比之前多一层判断力。

一、大多数人对 Claude Code 的理解停在了表面

现在网上流传的 Claude Code 分析，大部分聚焦在两件事：system prompt 写了什么，以及它调了哪些工具。

这两件事确实重要，但它们只是这个系统的皮肤。你拿到同样的 prompt，接上同样的工具，照着写一遍，出来的东西和 Claude Code 的体验差距会非常大。

差在哪？差在 prompt 背后有一整套编排机制，工具背后有一整条治理流水线，Agent 背后有一套分工和调度系统，生态扩展背后有一套让模型"知道自己能干什么"的感知通道。

这些东西加在一起，才构成了你用 Claude Code 时感受到的那个"稳"。这个稳不是来自某段神奇文案，是来自工程。

二、源码结构第一眼：这不是 CLI 工具，这是一个运行平台

从 src/ 目录的顶层看，Claude Code 至少有这些模块：入口层 entrypoints，常量与提示词 constants，工具定义 tools，运行时服务 services，命令系统 commands，UI 组件 components，协调器 coordinator，记忆系统 memdir，插件 plugins，Hook 系统 hooks，状态初始化 bootstrap，任务系统 tasks。

光看这个目录结构就能感觉到，这不是一个"把 LLM 接到终端里"的项目。

入口层更能说明问题。它有四个入口：CLI、初始化流程、MCP 模式、SDK。同一个 agent 运行时，可以服务四种不同的交互界面。这是平台化设计的标志。

命令系统也不是点缀。它注册了 /mcp、/memory、/permissions、/hooks、/plugin、/skills、/tasks、/plan、/review、/agents 等十几个系统级命令。而且命令系统不只加载内建命令，还统一加载 plugin commands、skill commands、bundled skills、动态 skills。所以命令系统本身就是生态的入口。

你回头想想，大部分开源 coding agent 的目录长什么样。通常是一个 main 文件，一个 prompt 文件，几个 tool 文件，一个 utils。Claude Code 的目录结构跟它们完全不在一个量级。这个量级差异不是为了好看，是因为它要解决的问题本身就更多。

三、Prompt 不是一段话，是一台组装机器

这可能是整个源码里最反直觉的部分。

大多数人以为 system prompt 是一大段固定文本，启动时塞进去就完了。Claude Code 完全不是这样。它的 system prompt 是由一个叫 getSystemPrompt() 的函数动态拼装出来的。

这个函数做的事，更像一个编排器。它先拼一组静态模块，再根据当前会话状态拼一组动态模块。

静态部分包括：身份定位（getSimpleIntroSection）、系统规范（getSimpleSystemSection）、做任务的哲学（getSimpleDoingTasksSection）、风险动作规范（getActionsSection）、工具

使用规范（getUsingYourToolsSection）、语气风格（getSimpleToneAndStyleSection）、输出效率（getOutputEfficiencySection）。

动态部分包括：session guidance、memory、环境信息、语言设置、输出风格、MCP instructions、scratchpad、function result clearing、summarize tool results、token budget、brief mode。

你可以这样理解：静态部分是这个 Agent 的"宪法"，不管什么会话都一样。动态部分是"当期政策"，根据你这次用的工具、连了哪些 MCP、用的什么语言、开没开 brief 模式来调整。

这里面有一个设计特别精巧。源码里有一个叫 SYSTEM_PROMPT_DYNAMIC_BOUNDARY 的标记，注释里写得很清楚：边界之前的内容尽量保持 cache 友好，边界之后是用户和会话特定的内容，不能乱改，否则会破坏缓存。

这是什么意思？这意味着 Anthropic 在管理 system prompt 的时候，已经在考虑 token 经济学了。边界之前的内容因为稳定，可以被 API 层缓存，不用每次都重新计算。边界之后的内容因为会变，所以放在后面，不影响前面的缓存命中。

大部分人写 prompt，根本不会想到这一层。但当你的产品每天要处理海量请求，每个 token 都有成本的时候，这种设计直接影响运营成本。

四、行为规范：Anthropic 怎么训一个 AI 工程师不乱来

Claude Code 有一个叫 getSimpleDoingTasksSection() 的模块，这个模块可能是整个系统里最不起眼但最有用的部分。

它做的事很简单：告诉模型什么该做，什么不该做。但它列得非常具体。

不要加用户没要求的功能。不要过度抽象。不要瞎重构。不要乱加注释和文档字符串。不要做不必要的错误处理和兜底逻辑。不要设计一堆面向未来的抽象。先读代码再改代码。不要轻易创建新文件。不要给时间估计。方法失败时先诊断原因再换策略。删除确认没用的东西，不搞兼容性垃圾。结果要如实汇报，不能假装自己测试过了。

如果你用过其他 coding agent，你一定遇到过这些问题：你让它改一个 bug，它顺手给你重构了半个文件。你让它加一个功能，它给你加了三层抽象和五个你没要的错误处理。你让它测一下，它说"测试通过了"，但其实它根本没跑。

这些问题的根源不是模型笨，是模型的行为没有被约束。Claude Code 解决这个问题的方式，不是靠更聪明的模型，是靠把行为规范写成制度。

风险动作规范也是同样的思路。getActionsSection() 定义了什么叫"需要确认的风险动作"：破坏性操作、难以回滚的操作、修改共享状态、对外可见的动作、上传到第三方工具。它还特别强调：不要用破坏性操作当捷径，遇到陌生状态先调查，merge conflict 和 lock file 不要粗暴删除。

这种设计的思路是把 blast radius 意识编进系统。你不能指望一个 AI 在每次做决定时都自己想到"这个操作的爆炸半径是多大"。但你可以在系统层面告诉它哪些操作需要停下来确认。

工具使用规范更具体。读文件用 FileRead，不要用 cat/head/tail。改文件用 FileEdit，不要用 sed/awk。新建文件用 FileWrite，不要用 echo 重定向。搜文件用 Glob，搜内容用 Grep。Bash 只留给真正需要 shell 的场景。没有依赖关系的工具调用要并行。

这不是在说"你有这些工具"。这是在说"你必须用正确的方式使用这些工具"。Claude Code 用起来稳，和这套 tool usage grammar 关系很大。很多 agent 的不稳定，恰恰来自模型用错了工具，比如用 bash sed 去改代码，一个正则写错就崩了。

五、多 Agent 分工：为什么一个人做不好所有事

源码里确认了至少六个内建 Agent：General Purpose Agent、Explore Agent、Plan Agent、Verification Agent、Claude Code Guide Agent、Statusline Setup Agent。

这个设计选择不是为了炫技。它背后有一个很清晰的判断：让一个 Agent 同时做研究、规划、实现、验证，最后每件事都做不扎实。

Explore Agent 被设计成纯只读模式。不能创建文件，不能修改文件，不能删除文件，不能移动文件，不能写临时文件，不能用重定向写文件，不能运行任何改变系统状态的命令。它能用的工具只有 Glob、Grep、FileRead，Bash 也只允许 ls、git status、git log、git diff 这类读操作。它被故意裁剪成了一个 read-only specialist。

为什么要这么做？因为探索阶段如果不小心改了什么东西，后面的实现阶段就会出问题。把探索和实现的权限彻底隔离，是一种很朴素但很有效的安全设计。

Plan Agent 也是只读。它的职责是理解需求、探索代码库的模式和架构、输出逐步实现计划、列出关键实现文件。它被定义成 architect，不是 executor。规划和实现分开的好处是，规划阶段可以专心想清楚怎么做，不会因为急着写代码而跳过思考。

六、Verification Agent：整个系统里最值钱的设计

Verification Agent 的 prompt 在整个源码里可能是写得最狠的一个。

它的核心方向不是"确认实现看起来没问题"。它的方向是 try to break it，想办法搞坏它。

prompt 开头就点出了两类常见的验证失败模式。第一种叫 verification avoidance：只看代码不跑检查，写个 PASS 就走了。第二种叫被前 80% 迷惑：UI 看起来还行，测试也过了，就忽略剩下 20% 的问题。

然后 prompt 强制要求一系列验证动作：跑 build，跑测试套件，跑 linter 和类型检查。根据变更类型还有专项验证。前端改动要跑浏览器自动化或验证页面子资源。后端改动要用 curl 或 fetch 实测响应。CLI 改动要看 stdout、stderr 和 exit code。数据库迁移要测 up 和 down，还要测已有数据。重构也要测 public API surface 有没有变化。

更狠的是，它要求做 adversarial probes，主动去找边界情况和破绽。每个检查必须带实际执行的命令和观察到的输出。最后必须给出 VERDICT: PASS、FAIL 或 PARTIAL。

这个设计解决了 LLM 验证工作中最常见的问题："差不多就算了"。人类工程师做 code review 的时候也会犯这个毛病，但至少人有经验积累和职业压力。LLM 没有这些，你不在 prompt 里把验证标准写死，它真的会随便看两眼就说 PASS。

Claude Code 的做法是用 prompt 反制这种倾向。而且因为 Verification Agent 是独立角色，它和"写代码的那个 Agent"没有利益关联。写代码的 Agent 可能会倾向于觉得自己写得没问题，但验证 Agent 没有这个偏见，它的工作就是找问题。

这种"实现者和验证者分离"的思路，在传统软件工程里是常识，但在 AI Agent 系统里，大部分产品还没做到这一步。

七、Agent 调度链：14 步 pipeline 不是过度设计

一个子 Agent 从被触发到执行完成，在 Claude Code 里要经过一条完整的 pipeline：

主模型决定调用 Agent 工具。AgentTool.call() 解析输入。系统判断这是 teammate、fork、built-in、background 还是 remote。选择对应的 agent definition。构造 prompt messages。构造或继承 system prompt。组装工具池。创建 agent 专属的 ToolUseContext。注册 hooks、skills、MCP servers。调用 runAgent()。runAgent 内部再调用 query() 进入主循环。query 产出消息流。runAgent 记录 transcript，处理生命周期，清理资源。AgentTool 汇总结果或走异步通知。

这 14 步看起来多，但每一步都在解决一个具体问题。

其中 fork 和 normal path 的区分很有意思。当你 fork 一个子任务时，它会继承主线程的 system prompt 和完整对话上下文，工具集也尽量保持一致。为什么？为了让 API 请求的前缀保持字节级一致，从而复用主线程的 prompt cache。

这个细节很容易被忽略，但它反映了一种思维方式的差异。普通人做子 Agent 调度，想的是"子任务能跑起来就行"。Claude Code 想的是"子任务能跑起来，而且尽量复用主线程的缓存，不白烧 token"。

runAgent() 本身也不是简单的 wrapper。它要初始化 agent 专属的 MCP servers，过滤和克隆上下文消息，处理文件状态缓存，对只读 Agent 做内容瘦身，构造权限模式，组装工具，注册 frontmatter hooks，预加载 skills，创建中断控制器，执行 SubagentStart hooks。执行完之后还要记录 transcript，清理 MCP 连接、hooks、性能追踪、todo、bash tasks 等资源。

这就是为什么文章开头说 Claude Code 更像一个运行平台。一个子 Agent 的启动和关闭，涉及的状态管理工作量，已经接近一个小型服务的生命周期管理了。

八、工具执行不是一步到位，是一条治理流水线

当模型决定调用一个工具时，Claude Code 不是直接执行对应的函数。实际链路是这样的：

先找到对应的 tool definition。解析 MCP metadata。用 Zod schema 做输入校验。跑 tool 自己的 validateInput。如果是 Bash 命令，还要跑一个 speculative classifier check 来预判风险。然后运行 PreToolUse hooks。解析 hook 返回的权限结果。走正式的 permission 决策流程。根据 permission 决策可能再次修正输入。到这里才真正执行 tool.call()。执行完之后记录 analytics、tracing 和

OTel。然后运行 PostToolUse hooks。处理结构化输出。如果失败了，还有 PostToolUseFailure hooks。

这条链路里最有意思的是 Hook 系统。PreToolUse Hook 不只能记日志，它能返回 message、blockingError、updatedInput、permissionBehavior、preventContinuation、stopReason、additionalContexts。也就是说 Hook 能改写输入，能直接放行或拒绝，能阻止后续流程，能补充上下文信息。

但 Hook 的权力也不是无限的。resolveHookPermissionDecision() 这个函数定义了一个关键规则：Hook 说 allow，不一定能绕过系统设置里的 deny/ask 规则。如果工具本身要求用户交互，而 Hook 没有提供替代输入，仍然要走统一的权限流程。Hook 说 deny 则直接生效。

这个设计很成熟。Hook 有足够的表达力来做运行时策略，但它不能绕开核心安全模型。强大但受控，这是工程成熟度的体现。

PostToolUse Hook 也不只是收尾工作。工具成功执行后，Hook 还能追加消息、注入额外上下文、阻断后续流程、更新 MCP 工具输出。工具执行失败后，Hook 能补充失败上下文、发阻断说明、给用户恢复线索。

整条链路的设计哲学是一样的：默认假设事情会出问题。模型可能乱传参数（所以有输入校验），操作可能有风险（所以有 speculative classifier 和权限决策），执行可能失败（所以有失败 hook），成功了也可能需要额外处理（所以有 post hook）。

你提前把这些"可能出问题"的地方都考虑到了，系统才会抗打。你假设一切正常，线上就会教你做人。

九、Skill、Plugin、MCP：生态的关键是模型"知道"

Claude Code 有三套扩展机制：Skill、Plugin、MCP。

Skill 不是文档，是 workflow package。它的形态是带 frontmatter metadata 的 markdown prompt bundle，可以声明 allowed-tools，可以按需注入当前上下文，可以把重复的工作流压缩成可复用的能力包。系统要求模型在任务匹配到某个 skill 时，必须调用 Skill tool 执行，不能只是提到这个 skill 而不执行。

Plugin 比 Skill 更重。它可以提供 markdown commands、SKILL.md 目录、commandsMetadata、userConfig、shell frontmatter、allowed-tools、model 和 effort hints、user-invocable 标记、disable-model-invocation 标记，还支持运行时变量替换。Plugin 不是普通的 CLI 插件，它是模型行为层面的扩展单元。

MCP 不只是工具桥。从 prompts.ts 里可以看到，当 MCP server 连接上来的时候，如果 server 提供了 instructions，这些 instructions 会被拼进 system prompt。也就是说 MCP 能同时给模型两样东西：新工具，以及怎么用这些工具的说明。

这三套机制有一个共同点：它们都不只是"挂载到系统上"。它们会通过 skills 列表、agent 列表、MCP instructions、session-specific guidance、command integration 这些通道，让模型感知到自己当前有哪些扩展能力、什么时候该用、怎么用。

很多平台也有插件系统，也有工具市场，但模型本身不知道这些东西存在。这就像你给一个人配了一整套专业工具箱，但他不知道箱子里有什么，也不知道什么时候该打开。Claude Code 的做法是把工具箱的清单和使用说明都放到模型看得见的地方。这才是生态真正能发挥作用的前提。

十、"怎么写给子 Agent 的 prompt"这一节很容易被忽略，但很有用

AgentTool/prompt.ts 里有一节专门教模型怎么给子 Agent 写 prompt。

它说：fresh agent 没有上下文，你要像给新来的同事做 briefing 一样写 prompt。说明目标和原因。说明你已经排除了什么。提供足够的背景让它能做判断。如果你只要一个短答案，明确说。不要把理解任务的工作外包给子 agent。不要写"基于你的发现再去修 bug"这种偷懒 prompt。应该给到具体的 file path、行号、具体改动要求。

这段规则在做什么？在限制"懒 delegation"。

多 Agent 系统里最常见的失败模式就是：主 Agent 把一个模糊的任务丢给子 Agent，子 Agent 理解偏了，做出来的东西不对，主 Agent 拿到结果也不知道问题出在哪。

Claude Code 通过 prompt 强制要求主 Agent 承担 synthesis 责任。你不能偷懒，你得把任务想清楚了再派出去。这种约束看起来增加了主 Agent 的负担，但实际上大幅减少了子 Agent 返工的概率。

十一、上下文经济学：它怎么把 token 当钱花

整个源码里有大量设计围绕一个主题：上下文是稀缺资源，不是免费空气。

system prompt 的静态/动态边界，前面说过了，是为了缓存。fork path 的 cache-identical prefix 设计，也说过了，是为了复用主线程缓存。

还有一些更细的设计。Skill 是按需注入的，不是一开始就全部塞进去。MCP instructions 是根据当前连接状态注入的，没连上的 server 的说明不会占上下文空间。有 function result clearing 机制，有 summarize tool results 机制，有 compact 和 transcript 机制，有 resume 机制。

这些机制加起来在做一件事：在有限的上下文窗口里，尽量装最有用的信息，尽量减少重复和冗余，尽量让缓存命中。

对于做 Demo 的人来说，上下文管理不是问题，因为 Demo 跑几次就结束了。但对于做产品的人来说，上下文经济学直接关系到成本和体验。你的系统每天处理几万次请求，每次请求的 system prompt 有几千个 token，缓存命中率提高 10%，一个月下来省的钱可能够你多雇一个人。

十二、产品化的最后一公里：生命周期管理

runAgent() 里有很多不起眼但很说明问题的代码：recordSidechainTranscript()、writeAgentMetadata()、registerPerfettoAgent()、cleanupAgentTracking()、killShellTasksForAgent()、清理 session hooks、清理克隆的文件状态、清理 todos entry。

后台 Agent 有独立的 abort controller，可以在后台持续运行，完成后通过 notification 回到主线程，支持自动 summarization。前台 Agent 可以在执行过程中被转成后台运行，有 progress tracking。

这些功能单独看都不惊艳。但它们加在一起，说明 Anthropic 不是只管"让 Agent 跑起来"，而是把 transcript 记录、性能追踪、资源清理、会话恢复、前后台切换全都当成了运行时生命周期的正式组成部分。

大部分 Agent 系统在第一天跑得挺好。问题出在第二天，第三天，第一百天。任务中断了怎么续？脏状态怎么清？子 Agent 的 shell 进程没杀掉怎么办？MCP 连接泄漏了怎么办？这些问题不解决，产品就只能是 Demo。

Claude Code 对这些问题都有明确的处理路径。这就是为什么它用起来更像一个正式产品，而不是一个很聪明的 prototype。

你能从这里面学到什么

把这些拆完之后，回头看，Claude Code 在做的事其实可以概括成几条设计原则。

第一条，不要信任模型的自觉性。好行为要写成制度，不要依赖模型临场发挥。你希望模型先读代码再改代码，就把这条规则写进 prompt。你希望模型不要乱加功能，就把这条规则写进 prompt。你希望模型遇到风险操作停下来确认，就在 runtime 层加权限检查。

第二条，把角色拆开。至少把"做事的人"和"验收的人"分开。哪怕现在条件有限，只用同一个模型，把职责拆开也会有明显改善。因为同一个 Agent 既实现又验证，天然会倾向于觉得自己做得没问题。

第三条，工具调用要有治理。不是模型说调就调，中间要有输入校验、权限检查、风险预判。执行完也不是结束，要有 post-processing 和失败处理。这层治理决定了系统在异常情况下的表现。

第四条，上下文是预算。每个 token 都有成本，每条信息都占空间。能缓存的要缓存，能按需加载的不要一开始就塞进去，能压缩的要压缩。做 Demo 可以不在乎这些，做产品必须在乎。

第五条，生态的关键是模型感知。你给系统接了十个插件，但模型不知道什么时候该用哪个，那这十个插件就等于不存在。扩展机制的最后一步，是让模型看到自己的能力清单，知道什么场景用什么能力。

这五条原则不只适用于 coding agent，也适用于几乎所有需要 LLM 做复杂任务的系统。Claude Code 的价值不在于某个具体实现，而在于它用工程实践验证了这些原则确实有效。

你不需要复刻它的全部。从最薄弱的环节开始补，每补一层，系统的"手感"就会好一个台阶。

一句话摘要：拆了 Claude Code 4756 个源码文件后发现，它的秘密不在 prompt 里，在一整套把行为制度、工具治理、Agent 分工、上下文经济学和生

Want to publish your own Article?
Upgrade to Premium

Xiao Tan
@tvytlx

Follow

prev senior fullstack engineer, now indie maker the world is a sandbox game that no one knows about.